Singly Linked List –
Get N^{TH} NODE
–Space Complexity and Algorithm.

Program :

```
int getNthNode(Node **head, int node)
{
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return -1;
    }
    if (node < 1 || node > lengthOfList(head))
    {
        cout << "Error: Invalid index." << endl;
        return -1;
    }
    else
    {
        Node *current = *head;
        int i = 1;
        while (i < node)
        {
            current = current->next;
            i++;
        }
        return current->data;
    }
}
…..
       case 18:
        {
            cout << "Nth node from beginning , where 1 is the first node(1 based
indexing):" << endl;
            int n;
```

```
            cout << "Enter the node: ";
            cin >> n;
            cout << "The " << n << "th node is from the beginning: " <<
getNthNode(&head, n) << endl;
            break;
        }
```

Space Complexity Analysis

1. Input Space Complexity

A. Parameter – Only Definition

The input space complexity refers to the space required for the input parameters of the function.

In this case, the function takes:

- ***A pointer to a pointer (Node ** head), which requires a constant amount of space, O(1).***
- ***An integer (int node), which also requires a constant amount of space, O(1).***

Thus, the total input space complexity is: O(1).

This view only considers the memory for the parameters passed to the function. The parameters are `Node **head` (a pointer-to-a-pointer) and `int node` (an integer), both of which have a **fixed, constant size**. Therefore, the input space complexity is **O(1)**.

• Node **head – one pointer (machine word)

• int node – one integer (machine word) → Θ(1) = O(1)

B. Full – Data (inclusive) Definition:

This more common definition includes the entire data structure. If the linked list has **n nodes**, the space required to store that list is proportional to `n`. Therefore, the input space complexity is **Θ(n)**.

• Same two scalars as above (Θ(1)), plus

• Every list node reachable from *head when the call starts.

If the list contains n nodes, they occupy Θ(n) bytes.

→ Θ(n) = O(n)

2. *Auxiliary Space Complexity*

Auxiliary space complexity refers to the extra space used by the algorithm that is not part of the input. In the case of the getNthNode function, the auxiliary space is determined by the variables declared within the function:

- ***The function uses a pointer Node * current to traverse the linked list, which requires O(1) space.***

- ***An integer variable int i is used to keep track of the current index during traversal, which also requires O(1) space.***

Since both of these variables occupy a constant amount of space regardless of the size of the linked list, the auxiliary space complexity is: O(1)

Auxiliary space is the extra memory an algorithm uses, not counting the input. For this function:

1. **`lengthOfList(...)`**: This function calls the `lengthOfList` helper function. As analyzed previously, this helper function has its own auxiliary space complexity of Θ(1) (it uses one pointer and one integer).
2. **`Node *current` and `int i`**: The `getNthNode` function itself declares one pointer and one integer to traverse the list. These variables also occupy a constant amount of memory.
3. **Iteration**: The `while` loop reuses these variables without allocating any new memory.

The total extra memory is the sum of the constant space used by `lengthOfList` and the constant space used by the local variables in `getNthNode`. The sum of several constants is still a constant.

Therefore, the auxiliary space complexity is **Θ(1)**.

Auxiliary (Extra) Space Complexity

What the algorithm itself allocates in addition to its input:

- **Local stack variables in getNthNode:**
  - **Node *current**
  - **int i**
  - **a few compiler-generated temporaries**

  → Θ(1)
- **The helper call lengthOfList(head) (iterative version analysed earlier):**
  - **Uses only Θ(1) extra space.**
- **No dynamic-heap allocations, no recursion.**

→ Total auxiliary space: Θ(1) = O(1)

$Algorithm$

$GetNthNode(START, NODE)$

$1.\ [START\ is\ NULL?]\ if\ START = NULL, then:$

$\quad Write: ``Error: List\ is\ Empty.``$

$\quad Return\ -1\ and\ exit.$

$[END\ of\ If\ structure]$

$2.\ if\ NODE < 1\ OR\ NODE > lengthOfList(START)\ , then:$

$\quad Write: ``Error: Invalid\ Index.``$

$\quad Return\ -1\ and\ Exit.$

$Else$

$\quad SET\ CURRPTR := START\quad [CURRPTR\ will\ point\ to\ Starting\ NODE]$

$\quad SET\ Count := 1$

$\quad Repeat\ while\ Count < NODE\ , then:$

$\quad\quad SET\ CURRPTR := LINK[CURRPTR]$

$\quad\quad SET\ COUNT := COUNT + 1$

$\quad [END\ OF\ Loop]$

$\quad Return\ INFO[CURRPTR]$

$[END\ of\ If\ structure]$

[$Translation\ of\ ``lengthOfList(START)``\ Operation$]

$lengthOfList(START)$

1. $SET\ Count := 0$
2. $SET\ CURRPTR := START$ [$CURRPTR\ will\ point\ to\ Starting\ NODE$]
3. $Repeat\ while\ CURRPTR\ \neq NULL\ , then$:

    $SET\ Count := Count + 1$ [$Update\ and\ increment\ COUNT$]

    $SET\ CURRPTR := LINK[CURRPTR]$ [$CURRPTR\ will\ now\ point\ to\ Next\ NODE$]

   [$End\ of\ LOOP\ structure$]

4. $RETURN\ Count$.

[$End\ of\ Algorithm$]

[$End\ of\ Algorithm$]